

```

++*rhs.use; // 递增右侧运算对象的引用计数
if (--*use == 0) { // 然后递减本对象的引用计数
    delete ps; // 如果没有其他用户
    delete use; // 释放本对象分配的成员
}
ps = rhs.ps; // 将数据从 rhs 拷贝到本对象
i = rhs.i;
use = rhs.use;
return *this; // 返回本对象
}

```

13.2.2 节练习

练习 13.27: 定义你自己的使用引用计数版本的 HasPtr。

练习 13.28: 给定下面的类，为其实现一个默认构造函数和必要的拷贝控制成员。

<pre> (a) class TreeNode { private: std::string value; int count; TreeNode *left; TreeNode *right; }; </pre>	<pre> (b) class BinStrTree { private: TreeNode *root; }; </pre>
--	---

13.3 交换操作

除了定义拷贝控制成员，管理资源的类通常还定义一个名为 swap 的函数（参见 9.2.5 节，第 303 页）。对于那些与重排元素顺序的算法（参见 10.2.3 节，第 342 页）一起使用的类，定义 swap 是非常重要的。这类算法在需要交换两个元素时会调用 swap。

如果一个类定义了自己的 swap，那么算法将使用类自定义版本。否则，算法将使用标准库定义的 swap。虽然与往常一样我们不知道 swap 是如何实现的，但理论上很容易理解，为了交换两个对象我们需要进行一次拷贝和两次赋值。例如，交换两个类值 HasPtr 对象（参见 13.2.1 节，第 453 页）的代码可能像下面这样：

```

HasPtr temp = v1; // 创建 v1 的值的一个临时副本
v1 = v2; // 将 v2 的值赋予 v1
v2 = temp; // 将保存的 v1 的值赋予 v2

```

这段代码将原来 v1 中的 string 拷贝了两次——第一次是 HasPtr 的拷贝构造函数将 v1 拷贝给 temp，第二次是赋值运算符将 temp 赋予 v2。将 v2 赋予 v1 的语句还拷贝了原来 v2 中的 string。如我们所见，拷贝一个类值的 HasPtr 会分配一个新 string 并将其拷贝到 HasPtr 指向的位置。

理论上，这些内存分配都是不必要的。我们更希望 swap 交换指针，而不是分配 string 的新副本。即，我们希望这样交换两个 HasPtr：

```

string *temp = v1.ps; // 为 v1.ps 中的指针创建一个副本
v1.ps = v2.ps; // 将 v2.ps 中的指针赋予 v1.ps
v2.ps = temp; // 将保存的 v1.ps 中原来的指针赋予 v2.ps

```

编写我们自己的 swap 函数

可以在我们的类上定义一个自己版本的 swap 来重载 swap 的默认行为。swap 的典型实现如下：

```
class HasPtr {
    friend void swap(HasPtr&, HasPtr&);
    // 其他成员定义，与 13.2.1 节（第 453 页）中一样
};
inline
void swap(HasPtr &lhs, HasPtr &rhs)
{
    using std::swap;
    swap(lhs.ps, rhs.ps);    // 交换指针，而不是 string 数据
    swap(lhs.i, rhs.i);      // 交换 int 成员
}
```

我们首先将 swap 定义为 friend，以便能访问 HasPtr 的（private 的）数据成员。由于 swap 的存在就是为了优化代码，我们将其声明为 inline 函数（参见 6.5.2 节，第 213 页）。swap 的函数体对给定对象的每个数据成员调用 swap。我们首先 swap 绑定到 rhs 和 lhs 的对象的指针成员，然后是 int 成员。



与拷贝控制成员不同，swap 并不是必要的。但是，对于分配了资源的类，定义 swap 可能是一种很重要的优化手段。



swap 函数应该调用 swap，而不是 std::swap

此代码中有一个很重要的微妙之处：虽然这一点在这个特殊的例子中并不重要，但在一般情况下它非常重要——swap 函数中调用的 swap 不是 std::swap。在本例中，数据成员是内置类型的，而内置类型是没有特定版本的 swap 的，所以在本例中，对 swap 的调用会调用标准库 std::swap。

但是，如果一个类的成员有自己类型特定的 swap 函数，调用 std::swap 就是错误的了。例如，假定我们有另一个命名为 Foo 的类，它有一个类型为 HasPtr 的成员 h。如果我们未定义 Foo 版本的 swap，那么就会使用标准库版本的 swap。如我们所见，标准库 swap 对 HasPtr 管理的 string 进行了不必要的拷贝。

518

我们可以为 Foo 编写一个 swap 函数，来避免这些拷贝。但是，如果这样编写 Foo 版本的 swap：

```
void swap(Foo &lhs, Foo &rhs)
{
    // 错误：这个函数使用了标准库版本的 swap，而不是 HasPtr 版本
    std::swap(lhs.h, rhs.h);
    // 交换类型 Foo 的其他成员
}
```

此编码会编译通过，且正常运行。但是，使用此版本与简单使用默认版本的 swap 没有任何性能差异。问题在于我们显式地调用了标准库版本的 swap。但是，我们不希望使用 std 中的版本，我们希望调用为 HasPtr 对象定义的版本。

正确的 swap 函数如下所示：

```
void swap(Foo &lhs, Foo &rhs)
```

```

{
    using std::swap;
    swap(lhs.h, rhs.h); // 使用 HasPtr 版本的 swap
    // 交换类型 Foo 的其他成员
}

```

每个 swap 调用应该都是未加限定的。即，每个调用都应该是 swap，而不是 std::swap。如果存在类型特定的 swap 版本，其匹配程度会优于 std 中定义的版本，原因我们将在 16.3 节（第 616 页）中进行解释。因此，如果存在类型特定的 swap 版本，swap 调用会与之匹配。如果不存在类型特定的版本，则会使用 std 中的版本（假定作用域中有 using 声明）。

非常仔细的读者可能会奇怪为什么 swap 函数中的 using 声明没有隐藏 HasPtr 版本 swap 的声明（参见 6.4.1 节，第 210 页）。我们将在 18.2.3 节（第 706 页）中解释为什么这段代码能正常工作。

在赋值运算符中使用 swap

定义 swap 的类通常用 swap 来定义它们的赋值运算符。这些运算符使用了一种名为拷贝并交换（copy and swap）的技术。这种技术将左侧运算对象与右侧运算对象的一个副本进行交换：

```

// 注意 rhs 是按值传递的，意味着 HasPtr 的拷贝构造函数
// 将右侧运算对象中的 string 拷贝到 rhs
HasPtr& HasPtr::operator=(HasPtr rhs)
{
    // 交换左侧运算对象和局部变量 rhs 的内容
    swap(*this, rhs); // rhs 现在指向本对象曾经使用的内存
    return *this;     // rhs 被销毁，从而 delete 了 rhs 中的指针
}

```

在这个版本的赋值运算符中，参数并不是一个引用，我们将右侧运算对象以传值方式传递给了赋值运算符。因此，rhs 是右侧运算对象的一个副本。参数传递时拷贝 HasPtr 的操作会分配该对象的 string 的一个新副本。

519

在赋值运算符的函数体中，我们调用 swap 来交换 rhs 和 *this 中的数据成员。这个调用将左侧运算对象中原来保存的指针存入 rhs 中，并将 rhs 中原来的指针存入 *this 中。因此，在 swap 调用之后，*this 中的指针成员将指向新分配的 string——右侧运算对象中 string 的一个副本。

当赋值运算符结束时，rhs 被销毁，HasPtr 的析构函数将执行。此析构函数 delete rhs 现在指向的内存，即，释放掉左侧运算对象中原来的内存。

这个技术的有趣之处是它自动处理了自赋值情况且天然就是异常安全的。它通过在改变左侧运算对象之前拷贝右侧运算对象保证了自赋值的正确，这与我们在原来的赋值运算符中使用的方法是一致的（参见 13.2.1 节，第 453 页）。它保证异常安全的方法也与原来的赋值运算符实现一样。代码中唯一可能抛出异常的是拷贝构造函数中的 new 表达式。如果真发生了异常，它也会在我们改变左侧运算对象之前发生。



使用拷贝和交换的赋值运算符自动就是异常安全的，且能正确处理自赋值。

13.3 节练习

练习 13.29: 解释 `swap(HasPtr&, HasPtr&)` 中对 `swap` 的调用不会导致递归循环。

练习 13.30: 为你的类值版本的 `HasPtr` 编写 `swap` 函数, 并测试它。为你的 `swap` 函数添加一个打印语句, 指出函数什么时候执行。

练习 13.31: 为你的 `HasPtr` 类定义一个 `<` 运算符, 并定义一个 `HasPtr` 的 `vector`。为这个 `vector` 添加一些元素, 并对它执行 `sort`。注意何时会调用 `swap`。

练习 13.32: 类指针的 `HasPtr` 版本会从 `swap` 函数受益吗? 如果会, 得到了什么益处? 如果不是, 为什么?

13.4 拷贝控制示例

虽然通常来说分配资源的类更需要拷贝控制, 但资源管理并不是一个类需要定义自己的拷贝控制成员的唯一原因。一些类也需要拷贝控制成员的帮助来进行簿记工作或其他操作。

作为类需要拷贝控制来进行簿记操作的例子, 我们将概述两个类的设计, 这两个类可能用于邮件处理应用中。两个类命名为 `Message` 和 `Folder`, 分别表示电子邮件 (或者其他类型的) 消息和消息目录。每个 `Message` 对象可以出现在多个 `Folder` 中。但是, 任意给定的 `Message` 的内容只有一个副本。这样, 如果一条 `Message` 的内容被改变, 则我们从它所在的任何 `Folder` 来浏览此 `Message` 时, 都会看到改变后的内容。

为了记录 `Message` 位于哪些 `Folder` 中, 每个 `Message` 都会保存一个它所在 `Folder` 的指针的 `set`, 同样的, 每个 `Folder` 都保存一个它包含的 `Message` 的指针的 `set`。图 13.1 说明了这种设计思路。

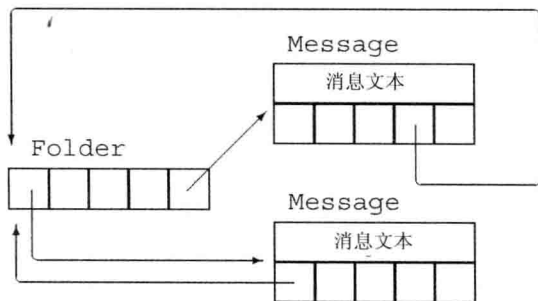


图 13.1: `Message` 和 `Folder` 类设计

我们的 `Message` 类会提供 `save` 和 `remove` 操作, 来向一个给定 `Folder` 添加一条 `Message` 或是从中删除一条 `Message`。为了创建一个新的 `Message`, 我们会指明消息内容, 但不会指出 `Folder`。为了将一条 `Message` 放到一个特定 `Folder` 中, 我们必须调用 `save`。

当我们拷贝一个 `Message` 时, 副本和原对象将是不同的 `Message` 对象, 但两个 `Message` 都出现在相同的 `Folder` 中。因此, 拷贝 `Message` 的操作包括消息内容和 `Folder` 指针 `set` 的拷贝。而且, 我们必须在每个包含此消息的 `Folder` 中都添加一个指向新创建的 `Message` 的指针。

当我们销毁一个 `Message` 时, 它将不复存在。因此, 我们必须从包含此消息的所有